

CtrlFuzz: A controllable diffusion-based fuzz testing for deep neural networks via coverage-aware manifold guidance

Aoshuang Ye^{a,b}, Shilin Zhang^a, Runze Yan^a, Jianpeng Ke^c, Fei Zhu^a, Benxiao Tang^{a,*}

^a School of Computer Science and Artificial Intelligence, Wuhan Textile University, Wuhan, 430200, China

^b Advanced Cryptography and System Security Key Laboratory of Sichuan Province, Sichuan, China

^c School of Cyber Science and Engineering, Wuhan University, Wuhan, 430072, China

ARTICLE INFO

Keywords:

Deep learning testing
Coverage-guided fuzzing
Manifold learning

ABSTRACT

Context: Deep neural networks (DNNs) have been extensively deployed in safety-critical applications. Nevertheless, the inherent vulnerability to subtle perturbations of inputs constitutes serious risks to the reliability of DNN-based systems. While mutation-based coverage-guided fuzzing (CGF) ensures test oracle through deliberately limited perturbations, it struggles to obtain diverse and sparse test cases. Conversely, generation-based CGF is able to create more diverse test cases aligned with data distribution but lacks precise controllability.

Objective: To refine the controllability and effectiveness of CGF in DNN testing, we aim to design a framework that is capable of generating realistic test cases with fine-grained control, while systematically exploring model vulnerabilities through a manifold-aware coverage criterion.

Method: In this paper, we propose *CtrlFuzz*, a manifold coverage-guided controllable diffusion framework for testing DNNs. CtrlFuzz leverages manifold learning to embed high-dimensional inputs into a lower-dimensional Euclidean space, preserving geometric structure. Based on this, we define a manifold coverage by quantifying the ratio between the distances from seed and the non-adversarial counterparts to class center. We further enhance the testing controllability via performing semantic decomposition on seed inputs. A customized diffusion model based on the U-Net structure integrates manifold coverage and semantic constraints into the denoising process, which allows to remain semantically natural while covering vulnerable regions.

Results: Experimental results on four popular datasets and ten benchmark DNN architectures demonstrate that CtrlFuzz (1) effectively maintains the semantic coherence of generated test cases, (2) achieves improved exploration of vulnerable manifold regions compared to existing CGF techniques, and (3) discovers significantly more error-inducing inputs on multiple model types.

Conclusion: CtrlFuzz introduces a novel manifold guiding and diffusion-based fuzzing for controllable test case synthesis. By enhancing both manifold coverage and controllability in CGF, CtrlFuzz improves the thoroughness and effectiveness of DNN testing, which offers a promising direction for future robustness evaluation frameworks.

1. Introduction

Deep neural networks (DNNs) have become core components in safety-critical systems such as autonomous driving [1], medical diagnosis [2], and industrial robot [3]. Nonetheless, due to the inherent defects introduced by overfitting in DNNs, the learning-based systems exhibit unstable or non-robust behavior when subjected to minor

perturbations in inputs, thereby compromising the reliability and trustworthiness in real-world deployment [4,5]. This systemic fragility not only limits the utility of DNNs but also raises crucial safety risks in real-world systems. For instance, in 2021, Tesla's autonomous driving system was reported failed to identify the obstacles under the influence of dense fog and flashing police lights, leading to a severe traffic

* Corresponding author.

E-mail addresses: asye@wtu.edu.cn (A. Ye), 2315363114@mail.wtu.edu.cn (S. Zhang), 2415063014@mail.wtu.edu.cn (R. Yan), kejianpeng@whu.edu.cn (J. Ke), zf@wtu.edu.cn (F. Zhu), bxtang@wtu.edu.cn (B. Tang).

<https://doi.org/10.1016/j.infsof.2025.107856>

Received 27 April 2025; Received in revised form 26 June 2025; Accepted 22 July 2025

Available online 30 July 2025

0950-5849/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

accident. Therefore, it is imperative to conduct systematic vulnerability assessment of DNN systems to prevent potential loss or degradation of functionality. To facilitate this, DNN testing aims to guarantee the target model maintain stable and trustworthy decision-making performance under diverse and adversarial input conditions [6].

DNN testing refers to evaluating the specific properties (e.g., robustness, fairness) of a neural networks under various circumstances by systematically creating representative test suite [7]. Thus, the effectiveness of DNN testing is heavily dependent on the quality of the test suite, which can be generally classified into manually labeled and synthetic. Manually labeled test data originates from real-world environments and is typically annotated through labor [8]. Whereas, synthetic data is automatically generated using mutators under pre-defined oracles. Irrespective of the origin, high-quality test suite satisfy three core elements: naturalness, diversity, and error-triggering capability [9].

Coverage-guided fuzzing (CGF) simultaneously maximizing the test coverage and defect exposure to support a more trustworthy software systems. The CGF in DNN testing can be concluded as two types: mutation-based and generation-based methods. Mutation-based CGF, such as *DeepHunter* [10], *NSGen* [11], and *DLRegion* [12], generate new test inputs by applying minor perturbations on seed inputs (e.g., adjustments to brightness, contrast, rotation, etc.), which typically constrain the level of perturbation via using L_p -norm bounds to preserve the semantic consistency. Nonetheless, although the L_p -norm bounds facilitate the reliability of test oracles, they also restrict generating out-of-distribution or near-distribution test inputs. Since the generated test cases are concentrated in the neighborhood of seed inputs, and those seeds are independently and identically distributed with the training data, it is difficult for mutation-based CGF to access test cases near the boundary of the data manifold, which are more capable of exposing model errors.

Whereas, generation-based CGF, which leverage generative models such as variational autoencoders (VAE) [13], generative adversarial networks (GAN) [14], and diffusion model [15] aim to produce inputs that are aligned with real-world data distribution. This approach enables the creation of more diverse and realistic test inputs while preserving oracle consistency. Nevertheless, generation-based CGF methods are highly dependent on the quality of the training data and involve inherently stochastic generation procedures [16], which makes it difficult to accurately control the properties of the synthesized test cases. Therefore, a critical challenge for generation-based CGF is to attain a balance between maintaining distributional consistency w.r.t. real-world data and achieving fine-grained control over the generation of test inputs.

Distribution-driven fuzz testing represents a practicable strategy [17–19], and it is widely assumed that data resides on a manifold. The hypothesis of manifold learning is that high-dimensional data is inherently structured on a low-dimensional latent manifold, where each local region can be approximated by Euclidean geometry [20]. Consequently, in the local neighborhoods, data distributions are amenable to representation through the Euclidean properties of the low-dimensional projections. Accordingly, our approach focuses on facilitating precise control of test case generation in the manifold space.

To tackle the aforementioned challenges, inspired by manifold hypothesis, we confine the mutation of seed inputs to a data-driven manifold, thereby achieving fine-granularity control and target direction during test generation process. Furthermore, we control the denoising process of the diffusion model along the manifold coverage to simultaneously retain the semantic coherence of the generated test cases and efficiently induce model misbehaviors. Our approach leverages manifold learning to model decision boundaries and semantic feature space within a Euclidean representation of the inputs, which ensures test inputs are capable of exposing model vulnerabilities while preserving continuity within data manifold.

In this paper, we propose *CtrlFuzz*¹, a manifold coverage-guided controllable diffusion based CGF tool to test deep neural networks. Specifically, we apply a manifold learning-based dimensionality reduction to obtain a low-dimensional embedding of the high-dimensional data manifold. This embedding preserves local geometric relationships of inputs, allowing test metrics to operate effectively within a Euclidean space. Subsequently, we exploit Euclidean distance-based test criterion to analyze the data distribution. By measuring the Euclidean distance between the seed and its furthest non-adversarial mutants (i.e., the most distant point that does not induce model misclassification), we define a manifold coverage criterion to guide test generation. To further enhance the precision, we perform semantic analysis on seed input to identify and isolate distinct semantic components, which allows a more precise constraint on target manifold region. Consequently, we introduce a novel diffusion model based on U-Net architecture, wherein manifold coverage constraints are integrated into the denoising process, which enables fine-grained control over the test generation. *CtrlFuzz* produces semantically coherent test cases that align with real-world data distributions, and systematically exposes model vulnerabilities by directing the perturbations to the target manifold. Our experimental results across four benchmark datasets and 10 benchmark deep neural networks demonstrate the efficacy of *CtrlFuzz* with a 91.5% of fault detection rate in uncovering model defects.

The contributions are summarized as follows:

- In *CtrlFuzz*, we propose a novel synchronization mechanism that combining the manifold coverage and semantic constraints with the denoising steps of U-Net, thereby enabling more precise control over the test case generation process.
- We innovatively propose a target manifold region-based coverage metric to quantify the progression of on-manifold guidance, which allows guiding the test case to a deliberately defined error-inducing manifold.
- Experiments conducted on 4 datasets and 10 DNN models demonstrate that our method outperforms existing approaches in defect exposure with up to 91.5% of fault detection rate and naturalness.

This paper is structured as follows: In Section 2, we introduce relevant concepts of DNN testing and etc. In Section 3, we detail the methodology of *CtrlFuzz*. In Section 4, we present the empirical study and the experimental results. For the following Section 5, the ablation study is discussed. To summarize, in Section 6, we conclude the contribution and findings of this work.

2. Related work

2.1. Coverage-guided fuzzing for DNNs

Mutation-based CGF. To explore model defects, mutation-based CGF apply minor modifications to inputs, which involves affine operations (rotation, scaling, translation), pixel-level transformation (brightness and contrast), and noising (Gaussian and salt-and-pepper noise) [21]. For instance, Xie et al. [10] proposed their work *DeepHunter* that utilizes affine transformation $\mathcal{G}$ and pixel value transformation $\mathcal{P}$ to mutate seed inputs under guidance of metamorphic relations. Odena et al. [7], in their work *TensorFuzz*, integrated nearest neighbor algorithm to mutate inputs via white noise to optimize the test coverage in fuzzing neural network. On the other hand, Huang et al. [11] utilizes neuron semantics as guidance during test generation in *NSGen*. Different from *DeepHunter*, a style transformation $\mathcal{T}$ is integrated in fuzzing tool. Furthermore, Tian et al. [22], in *DeepTest*, implemented the image transformation and weather filters to optimize neuron coverage for testing autonomous driving system. Then Yuan et al. [23]

¹ The artifact is released at <https://github.com/Konatazz/CtrlFuzz>.

introduced more mutators which includes weather filter and adversarial perturbation to facilitate a decision-based metamorphic testing.

To conclude, since the aforementioned methods use mutators like affine transformation, pixel transformation, and style transformation, it is difficult to explore inputs beyond the original data distribution, which affects the comprehensiveness of testing neural networks [24].

Generation-based CGF. Generation-based methods [16] utilize deep generative models, e.g., GANs [25], VAEs [26], and diffusion models [27], to generate test cases from the training data distribution. Zhang et al. [14], in their work *DeepRoad* used GANs to generate severe weather to uncover vulnerabilities of DNN-based auto-driving systems. Huang et al. [28] proposed a hierarchical distribution-aware testing tool. This method leverages genetic algorithms to optimize both the perceptual quality and the distributional novelty of test inputs, thereby producing more natural test data. Zhou et al. [5] introduced *DeepBillboard*, a framework for generating adversarial billboards in real-world scenarios. The adversarial test cases generated by *DeepBillboard* are capable of inducing potential steering errors in autonomous driving systems. Missaoui et al. [15] presented *GENFUZZER*, a diffusion-model-based testing framework. *GENFUZZER* integrates fidelity analysis and test coverage tracking to preserve realistic and high-coverage test cases, effectively exposing vulnerabilities in deep learning models.

Thus, the generation-based CGF overcome some limited aspects in mutation-based CGF (i.e., the generated tests are more sparse and aligned with training data), but still face challenges such as weak controllability and high computational demands. While existing researches have made progress in enhancing the effectiveness of DNN testing, systematically exploring model vulnerabilities while ensuring test qualities remains an crucial issue.

2.2. Coverage criteria

We categorize the coverage criteria in deep learning testing into two types. The first one is structural coverage, which evaluates test adequacy via measuring the extent to which structural components in a target deep learning model are covered. The second type is distribution-based coverage, which assesses the diversity of test cases based on the analysis of data distribution characteristics.

The most classical structural coverage in deep learning testing is neuron coverage, which proposed by Pei et al. in their work *DeepXplore* [29]. Then a family of neuron coverage including k-multisection neuron coverage (KMNC), neuron boundary coverage (NBC), and strong neuron activation coverage (SNAC) was evaluated in *DeepGauge* [30]. Yuan et al. [9] proposed a coverage metric that treats the hierarchical neuron outputs as an approximate mapping of the data distribution. Ji et al. [31] introduced a causality-based criterion that constructs a causal graph to capture the transmission among neurons, which enables the discovering distinction between adversarial inputs, such as adversarial examples and backdoor-triggered inputs. Moreover, Shi et al. [32] utilized layer-wise relevance propagation (LRP) to identify key neurons. They constructed a Markov model from the activation, then quantified differences in transition matrices via KL divergence and adopted this divergence as a test coverage metric. Although structural coverage are effective in indicating test adequacy, they are inherently constrained by the limited interpretability of neural network architectures and the internal states [33].

Recent researches have increasingly focused on distribution-based coverage metrics, which assess the testing process by analyzing the data distribution. *DistXplore* [34] was proposed to jointly consider distribution discrepancy and distribution diversity, aiming to ensure both statistical similarity to the training distribution and internal diversity within the test data. Kim et al. [35] introduced two types of surprise coverage: the likelihood-based Likelihood-based surprise coverage (LSC), which quantifies the surprise rate of test inputs via model likelihoods, and the distance-based surprise coverage (DSC),

which assesses input novelty via measuring feature-space distances relative to the training data. Additionally, *CIT4DNN* [36] proposed a combinatorial interaction testing framework based on input distribution coverage in latent space, which ensures comprehensive coverage of feature combinations in the input domain.

2.3. Manifold learning

According to the manifold hypothesis, high-dimensional data are typically assumed to lie on or near a low-dimensional manifold. Dimensionality reduction serves as a means to reveal the latent structure, presenting complexity reduction while striving to preserve the geometric properties of the original data. Common dimensionality reduction methods include Principal Component Analysis (PCA) [37], t-SNE [38], Isomap [39], Local Linear Embedding (LLE) [40], and Linear Discriminant Analysis (LDA) [41], etc. Although t-SNE is effective for visualizing data, it suffers from high computational cost. Moreover, t-SNE is designed to preserve local structure while providing limited insight into global geometric relationships. And methods such as Isomap and LLE aim to preserve geometric structure and local features through geodesic distances and local linear approximations respectively, the ability to model global structure is limited. In the complex multi-class classification task, the aforementioned methods fail to ensure both interpretability and consistency of the data distribution after projection. In contrast, LDA demonstrates superior performance in preserving class separability and structural coherence, which makes it a more appropriate choice for *CtrlFuzz*.

2.4. Diffusion model

Diffusion models achieve impressive performance in high-quality image generation by gradually adding noise to data and then reconstructing samples through a reverse denoising process [42]. Common diffusion models include Denoising Diffusion Probabilistic Models (DDPM) [43], Improved Denoising Diffusion Probabilistic Models (ID-PPM) [44]. DDPM applies noise perturbations in the form of a Markov chain and learns the denoising process via neural networks. While it generates high-quality outputs, the sampling speed is relatively slow. ID-PPM improves sampling efficiency by optimizing the loss function and accelerating the reverse diffusion process. Diffusion autoregressive models incorporate autoregressive structures, making the generation process more flexible.

3. Methodology

We propose a novel DNN fuzz testing approach based on manifold learning to achieve fine-grained control over the test case generation process. As illustrated in Fig. 1, the *CtrlFuzz* comprises four key steps: first, to employ dimensionality reduction, we project the test data manifold into a low-dimensional latent space, thereby constructing a latent feature distribution of test data. Second, we perform semantic decomposition on the seed inputs to accurately distinguish between foreground objects and background components. Third, a metric based on the Euclidean distance between sample feature vectors is defined within the low-dimensional space to measure distributional divergence across the manifold. A linear interpolation strategy is further designed to enable target-oriented guidance during test case evolution. Finally, we develop a diffusion-based generative framework built upon the U-Net architecture. By integrating coverage information and semantic constraints, the model enables controllable test generation that conforms to both structural diversity and semantic consistency.

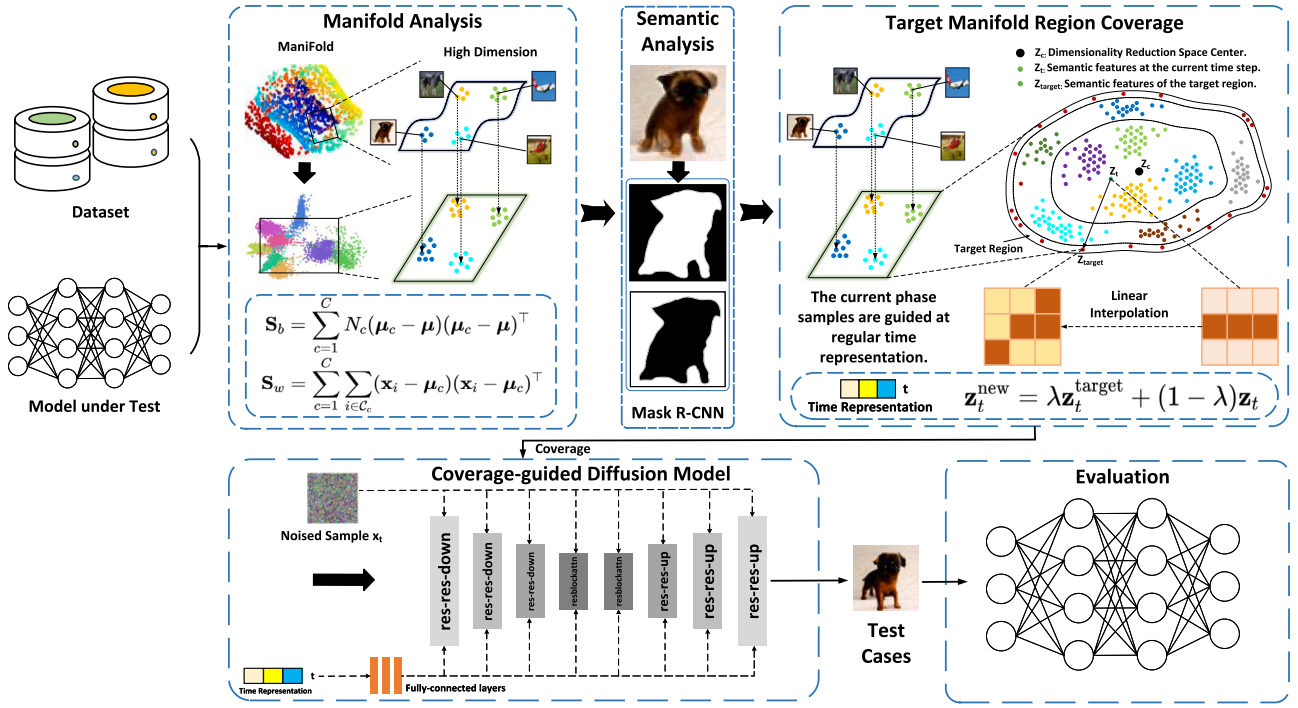


Fig. 1. Overview of CtrlFuzz.

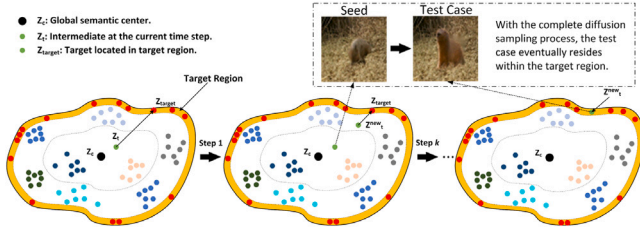


Fig. 2. The guiding process follows the diffusion time steps, achieving a step by step controllable generation to target manifold.

3.1. Manifold analysis

Manifold analysis derives low-dimensional representations from the high-dimensional space embedded data manifold to systematically explore the latent feature space nearing an initial seed input. This process establishes the foundational topology for subsequent target-oriented coverage guiding through dimensionality reduction and manifold coverage modeling. In *CtrlFuzz*, manifold analysis consists of the following steps: feature normalization, dimensionality reduction, and latent feature space analysis (see Fig. 2).

3.1.1. Feature normalization

Variability in feature scales from different dimensions can lead to certain features inappropriately influencing on subsequent manifold analyses which rely on distance-based calculations (e.g., t-SNE or UMAP). To tackle this issue, we employ the feature normalization to rescale all dimensions to a unified range, guaranteeing an equitable contribution from each feature during manifold analysis.

Consider a standardized dataset $D = \{(x_i, y_i)\}_{i=1}^N$, where each input $x_i \in \mathbb{R}^d$ is a d -dimensional feature vector and $y_i \in \{1, 2, \dots, C\}$ represents the class label respectively. For every feature dimension $j \in \{1, 2, \dots, d\}$, we perform normalization such that it achieves a mean of zero and a variance of one. In Eq. (1), the normalized value $\tilde{x}_{ij}$ for

the j th feature of input i is computed as:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \quad (1)$$

where $\mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij}$ and $\sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2}$ represent the population mean and standard deviation of the j th feature, respectively.

This step facilitates that all features reside on a comparable scale, thereby avoiding potential distortions during dimensionality reduction and enhancing the reliability of the learned low-dimensional projections.

3.1.2. Dimensionality reduction

In *CtrlFuzz*, we utilize LDA for the purpose of dimensionality reduction. LDA identifies an optimal subspace by maximizing the ratio of between-class scatter to within-class scatter [45], which preserves inter-class discriminability while compressing intra-class variability. The selection of LDA is based on considering computational efficiency and the requirements of multi-class scenarios, where the distinction between classes is critical for subsequent processing stages.

Let denote the normalized dataset with $X \in \mathbb{R}^{N \times d}$. In Eqs. (2) and (3), the matrix of between-class scatter S_b and the matrix of within-class scatter S_w are defined as:

$$S_b = \sum_{c=1}^C N_c (\mu_c - \mu)(\mu_c - \mu)^T, \quad (2)$$

$$S_w = \sum_{c=1}^C \sum_{i \in C_c} (x_i - \mu_c)(x_i - \mu_c)^T. \quad (3)$$

The N_c represents the number of inputs in class c , while μ_c and μ refer to the mean vectors specific to class c and overall, respectively. As Eq. (4) illustrates, the projection matrix of $W \in \mathbb{R}^{d \times k}$ is obtained by solving the generalized eigenvalue problem as follows:

$$S_b W = \lambda S_w W. \quad (4)$$

Here, the columns of W are aligned with the eigenvectors linked to the $k = \min(C - 1, d)$ highest eigenvalues. And in Eq. (5), the

Algorithm 1 Manifold-Guided Linear Interpolation**Require:**

z_i : Current semantic feature.
 $\mathcal{R} = \{R_1, R_2, R_3, R_4\}$: Predefined manifold regions.
 z_c : Target semantic center.
 λ : Interpolation factor ($\lambda \in [0, 1]$).
 T : Total number of generation steps.

Ensure:

z_i : Feature located in the target region.
1: $z_0 \leftarrow \text{Initialize_Semantic_Feature}()$ ▷ Initialize semantic feature of starting seed input.
2: $R_{\text{target}} \leftarrow \text{Set_Target_Region}(\mathcal{R})$ ▷ Select a semantic region based on test objective or vulnerability.
3: $z_c \leftarrow \text{Get_Region_Center}(R_{\text{target}})$ ▷ Retrieve the semantic center for the selected region.
4: **for** $t = 1$ **to** T **do**
5: $z_t \leftarrow \text{generate_step}(z_{t-1})$ ▷ Generate semantic feature at step t .
6: **if** $\text{distance}(z_t, R_{\text{target}}) > \epsilon$ **then**
7: $z_c \leftarrow \lambda \cdot z_{\text{target}} + (1 - \lambda) \cdot z_t$ ▷ Linearly interpolate toward target region center.
8: **end if**
9: **end for**
10: **return** z_t

Table 1Effect of the number of semantic regions R in TMRC.

Model	R = 2	R = 3	R = 4	R = 5	R = 6
ResNet34	24.5	61.5	88.5	90.0	87.5
VGG13_bn	27.0	50.5	87.0	86.5	88.0
DenseNet121	18.0	68.0	90.5	88.0	90.0
MobileNet_v2	23.5	47.0	89.0	90.5	89.0

dimensionality-reduced representation $Z \in \mathbb{R}^{N \times k}$ is then computed as:

$$Z = XW. \quad (5)$$

Given the assumptions of LDA, the projection is assumed to demonstrate a Lipschitz continuity. Specifically, when class manifolds are linearly separable with non-zero margins, a constant $\sigma > 0$ can be established such that for any pair of points $x_i, x_j \in \mathbb{R}^d$:

$$\|Z_i - Z_j\|_2 \leq \sigma \|x_i - x_j\|_2,$$

where Z_i, Z_j represent the LDA projections of x_i and x_j , respectively. This property guarantees that local neighborhoods in the original feature space are preserved in the reduced feature space, which justifies the role of LDA in following manifold-aware guidance step.

3.2. Latent feature space

Since we utilize a diffusion model to perform mutations on seed inputs, it is necessary to construct a semantic latent feature space around the seed in order to establish more precise guidance. Specifically, the entire training dataset forms a global semantic latent feature space, while the low-dimensional semantic features Z are used to define the semantic feature space.

Here, we implement the convex hull to approximate the semantic boundary of class c for $c \in \{1, 2, \dots, C\}$ in low-dimensional space. In this case, we define the convex hull as the minimal convex set that contains all feature points belonging to class c . Formally, for class c , let $\mathcal{Z}_c = \{z_i \in Z \mid y_i = c\}$ represent the semantic feature set corresponding to class c . In Eq. (6), the convex hull $K(\mathcal{Z}_c)$ of $\mathcal{Z}_c$ is defined as follows:

$$K(\mathcal{Z}_c) = \left\{ z \mid z = \sum_i \alpha_i z_i, \alpha_i \geq 0, \sum_i \alpha_i = 1, z_i \in \mathcal{Z}_c \right\}. \quad (6)$$

Therefore, we obtain the $K(\mathcal{Z}_c)$ as the decision boundary of class c , and any point within this boundary is considered semantically consistent with class c . By constructing the convex hull for each

class, we obtain a collection of class-specific semantic boundary sets: $\{K(\mathcal{Z}_1), K(\mathcal{Z}_2), \dots, K(\mathcal{Z}_C)\}$.

In *CtrlFuzz*, convex hulls are employed to define class-specific boundaries, which facilitates precise and interpretable neighborhood feature space. This approach presents that the performance of the target model is assessed explicitly by pre-defined semantic constraints, which is crucial for conducting systematic testing in multi-class classification tasks.

3.3. Semantic decomposition

In this section, a step of semantic decomposition intends to extract the specific regions of the seed input that are appropriate for further mutations in fuzzing. To facilitate this, we utilize a third-party semantic segmentation tool (i.e., Mask R-CNN) [46] to partition the seed input into foreground (object) and background. For an initial image I , we employ a semantic segmentation function $\text{Seg}(\cdot)$ to obtain the binary masks M_{fg} for the foreground and M_{bg} for the background, as defined in Eq. (7):

$$M_{\text{fg}}, M_{\text{bg}} = \text{Seg}(I), \quad (7)$$

where $M_{\text{fg}} \in \{0, 1\}^{H \times W}$ is defined as the binary mask that indicates the foreground region, and the value of 1 indicates an object pixel. Conversely, $M_{\text{bg}} = 1 - M_{\text{fg}}$ represents the background region of seed input which are the regions devoid of objects.

Subsequently, we isolate the foreground and background components from the input via element-wise masking, as demonstrated by Eq. (8):

$$I_{\text{fg}} = I \odot M_{\text{fg}}, \quad I_{\text{bg}} = I \odot M_{\text{bg}}, \quad (8)$$

Here, $\odot$ represents the Hadamard product, which is the element-wise multiplication. Through the process of Eqs. (7) and (8) show, *CtrlFuzz* is able to extract the foreground I_{fg} and background I_{bg} from a seed input. These constitute the foundation for the following steps in manifold-aware and region-specific mutation. This method facilitates fine-grained and interpretable analysis of semantic components, thereby enhancing the precision and control in test case mutation.

3.4. On-manifold guiding

To synthesis test cases that adhere to semantic constraints (introduced by decomposition) and reveal model vulnerabilities, we implement a manifold-guided control strategy into the diffusion process. This directs the generation along a low-dimensional manifold embedded in the high-dimensional data space, thereby establishing semantic in-

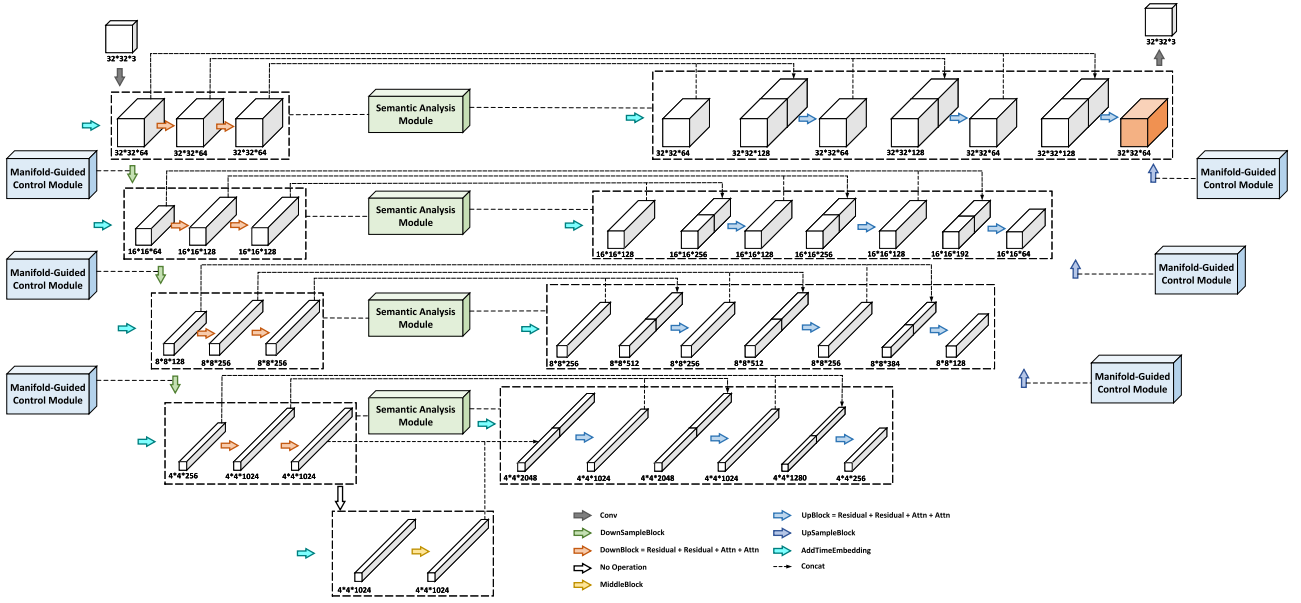


Fig. 3. U-Net structure in CtrlFuzz.

tegrity and realistic distribution. Consequently, this approach achieves accurate and controllable mutations. The detailed process is illustrated in Algorithm 1.

In Algorithm 1, the parameter R is set to 4, which is an empirical value. We designed a series of experiments to verify the rationality of this setting. In Table 1, each row represents the FDR in the outermost semantic region under different numbers of region divisions $R = \{2, 3, 4, 5, 6\}$. These semantic regions are constructed as concentric bands centered around the global semantic mean in the LDA-reduced space, defined by normalized Euclidean distance intervals. For example, when $R = 4$, the outermost region corresponds to the interval (0.95, 0.99); when $R = 5$, it becomes (0.99, 0.999); and when $R = 6$, it is (0.999, 0.9999). The results show that increasing R improves the granularity of semantic coverage and enhances fault detection capability, with peak performance observed at $R = 4$ to $R = 5$. Further increasing R leads to slight decline on performance. Increasing the number of R will further extend the number of steps. Therefore, choosing four is a relatively appropriate option.

During diffusion process, we adopt a projection strategy based on the LDA-reduced low-dimensional space. At fixed time intervals, the generated image features are projected into the low-dimensional semantic space. The core objective of this process is to ensure that the feature trajectories remain within the valid region of the manifold space under the guidance of semantic constraints.

At each time step t , we project the generated image feature $\mathbf{x}_t$ to derive the low-dimensional semantic feature $\mathbf{z}_t$, which is calculated using:

$$\mathbf{z}_t = \mathbf{x}_t W. \quad (9)$$

Given this, $\mathbf{x}_t \in \mathbb{R}^{N \times d}$ refers to the feature at each time step t , $W \in \mathbb{R}^{d \times k}$ represents the LDA transformation matrix, and $\mathbf{z}_t \in \mathbb{R}^{N \times k}$ is the low-dimensional semantic feature. At each projection step, the feature $\mathbf{x}_t$ is dynamically mapped into the reduced-dimensional space to ensure that the structure of the generated image features aligns with the semantic constraints of the manifold.

3.4.1. Target manifold region coverage

Several pioneer works have explored how to evaluate test adequacy on manifold. The k-section Manifold Combination Coverage (k-MCC) [47] and Input Distribution Coverage (IDC) [48] employ Variational Autoencoders (VAEs) to learn latent feature representations

of inputs. IDC concentrates on measuring the sufficiency of test-set coverage over the latent feature space. Thus, first learning the input manifold via a VAE, filtering out noise and out-of-distribution samples, and then computing feature-interaction coverage through symmetric-density partitioning and t-way combinatorial testing. Similarly, k-MCC maps inputs into the latent space via a VAE but discretizes each dimension into intervals. In CtrlFuzz, our goal is to generate error-triggering test cases that near the decision boundary of MuTs via guiding the diffusion process.

To facilitate this goal, we operate entirely within the low-dimensional semantic space $\mathbb{R}^k$ derived via LDA projection. At each time step t , the generated image features $\mathbf{x}_t \in \mathbb{R}^d$ are projected into this space as $\mathbf{z}_t = \mathbf{x}_t W$, where $W \in \mathbb{R}^{d \times k}$ is the learned LDA transformation.

Within this projected space, we partition the semantic manifold into several concentric regions R_1, R_2, R_3, R_4 centered around the global semantic mean $\mathbf{z}_c \in \mathbb{R}^k$. These regions correspond to increasing distances from the center, measured via the Euclidean norm in $\mathbb{R}^k$, and serve as semantic coverage targets. To quantify how well a generated $\mathbf{z}_t$ covers the manifold regions, we define the *Target Manifold Region Coverage* (TMRC) as:

$$TMRC_t = \frac{\|\mathbf{z}_t - \mathbf{z}_c\|_2}{\|\mathbf{z}_{\text{target}} - \mathbf{z}_c\|_2}. \quad (10)$$

Here, $\mathbf{z}_{\text{target}} \in \mathbb{R}^k$ is the nearest feature in the selected target region to $\mathbf{z}_t$. A TMRC value closer to 1 indicates the current sample has reached the semantic boundary of the intended region, while lower values imply under-coverage.

To guarantee that the generated samples sufficiently cover the target manifold region, as the algorithm 1 demonstrates, we introduce a linear interpolation strategy to guide the generation process. At each fixed time step, we interpolate between the current feature and the target feature, as shown in the equation below:

$$\mathbf{z}_t^{\text{new}} = \lambda \mathbf{z}_{\text{target}} + (1 - \lambda) \mathbf{z}_t. \quad (11)$$

Here, $\mathbf{z}_t^{\text{new}}$ is the adjusted semantic feature, $\mathbf{z}_{\text{target}}$ is the target region's semantic feature, and λ is a time-dependent interpolation factor. By controlling λ , we can flexibly steer the generation direction. Typically, λ is sampled from a uniform distribution in the range $\lambda \in [0, 1]$.

By projecting the intermediate features back into the semantic space at each step, we ensure that the generated feature gradually approach

and reach the target manifold region, which improves test diversity and validity.

Algorithm 2 Coverage-guided Diffusion Process

Require:

z_0 : Seed
 z_c : Class center
 z_{target} : Target manifold region
 T : Total number of diffusion steps
 W : LDA projection matrix
 M_{fg} : Foreground mask

Ensure:

z_t : target semantic feature
 1: $z_t \leftarrow z_0$
 2: **for** $t = 1$ to T **do**
 3: $z_t \leftarrow \text{Denoise}(z_t, t)$
 4: $\mathbf{x}_t \leftarrow \text{Decode}(z_t)$
 5: **if** $t \bmod k = 0$ **then**
 6: $\mathbf{z}_t \leftarrow \mathbf{x}_t W$
 7: $\text{TMRC}_t \leftarrow \frac{\|z_t - z_c\|_2}{\|z_{\text{target}} - z_c\|_2}$
 8: $\nabla z_t \leftarrow \nabla \mathcal{L}$
 9: $z_t \leftarrow z_t - \eta \cdot \nabla z_t$
 10: $z_t \leftarrow z_t \odot M_{\text{fg}}$
 11: **end if**
 12: **end for**
 13: **return** z_t

3.5. Coverage-guided diffusion model

The proposed *CtrlFuzz* is built on the foundation U-Net architecture, as illustrated in Fig. 3, which promotes the generated images are iteratively optimized to satisfy pre-defined semantic constraints and coverage guidance. To enhance the semantic diversity of the generated tests, we specifically design a residual attention module that follows the sequential structure: Residual Block $\rightarrow$ Residual Block $\rightarrow$ Class Attention Block $\rightarrow$ Channel Attention Block.

To address the gradient flow and the vanishing gradient issue typical of deep networks, two sequential residual blocks are initially employed. In the next step, the Class Attention Block is utilized to amplify class-specific features, enabling the model to concentrate on the most relevant object categories during generation process. Subsequently, the Channel Attention Block performs channel-wise weighting to the feature maps, further enhancing the feature representation by prioritizing the most informative channels while suppressing redundant information.

Algorithm 2 presents the proposed diffusion process guided by TMRC feedback. Every k steps, the generated image features are projected into the LDA-reduced low-dimensional space via a linear transformation matrix W . We then compute the TMRC, defined as the ratio between the current distance $\|z_t - z_c\|_2$ and the target boundary distance $\|z_{\text{target}} - z_c\|_2$. The foreground mask M_{fg} obtained from semantic segmentation is applied element-wise to the latent features. This enforces the generative process remains focused on semantically meaningful regions.

4. Experiments

We deployed our proposed *CtrlFuzz* on 10 popular models from 6 DNN families that are trained with 4 computer vision datasets. The implementation was based on Python 3.10.14, Opencv 4.10, Numpy 1.23.5, etc.

4.1. Experiment configurations

Datasets. We implement the target DNNs on four popular computer vision datasets: ImageNet(64×64) [49], TinyImageNet [50], CIFAR-10, and MNIST. ImageNet(64×64) is a resized version of the large-scale ImageNet dataset, which contains over one million images categorized into 1000 different classes, with images resized to 64×64 pixels. TinyImageNet is also a subset of ImageNet, which contains 200 categories, each class with 500 training images and 50 validation images, and all resized to 64×64 pixels. CIFAR-10 consists of 60,000 32×32 color images across 10 classes, with 6000 images per class, and is commonly used for benchmarking machine learning algorithms in image classification tasks. MNIST, composed of 70,000 28×28 grayscale images of handwritten digits, is split into 60,000 training images and 10,000 test images, covering 10 classes from 0 to 9, and has long been a standard dataset for evaluating image classification algorithms.

Model under Test. To perform thorough and systematic experiments, we employ 10 famous deep classifiers as the MuT, which includes ResNet34, ResNet50[51], VGG13_bn, VGG16_bn [52], DenseNet121, DenseNet161[53], Inception_v3[54], MobileNet_v2[55], LeNet-1, and LeNet-5. As shown in Table 2, the deepest network is DenseNet161, consisting of 161 layers and 26,482,570 parameters, while VGG16_bn has the largest number of parameters, totaling 33,646,666.

Evaluation Metrics. We demonstrate the effectiveness of *CtrlFuzz* through the evaluation of several key metrics that measure different aspects of model performance and generated sample quality. The chosen metrics are as follows:

- **FDR** (Fault Detection Rate) represents the proportion of test cases in a test suite that successfully induce errors in the model. It is critical for evaluating how well *CtrlFuzz*-generated test cases expose model weaknesses, particularly under adversarial or edge cases. A higher FDR indicates better model vulnerability detection.
- **PRDC** (Precision, Recall, Density, Coverage) [56] is a set of metrics used to evaluate the quality and diversity of the generated samples, focusing on how well they represent the real data distribution.
 - **Precision** measures the proportion of relevant generated samples that align with real data's semantic features.
 - **Recall** evaluates the ability of generated samples to cover a wide spectrum of the real data distribution.
 - **Density** refers to the concentration of generated samples in feature space, indicating the representations of dense regions of real data.
 - **Coverage** measures the extent of the generated samples cover the real data distribution, assuring diversity in the test cases.
- **SSIM** (Structural Similarity Index) [57] measures the similarity between generated and real samples, focusing on structural information, luminance, and contrast. A higher SSIM indicates that the generated samples closely resemble real data in terms of structure and key image properties.

Baselines. To demonstrate the efficacy of *CtrlFuzz*, we select several prevalent coverage metrics including NC, NBC, SNAC, TKNC, KMNC and IDC [30,48]. For comparative experiment with generative model-based testing tool, we implement TIGs [58]. The settings for various methodologies are as follows: the threshold of NC stands at 0.4, 0.5 and 0.6, the k in TKNC is set as 1, 3 and 5, the k in KMNC is set as 100 and 1000.

Training Details. We trained all models for 235 epochs on a system equipped with an Intel(R) Core(TM) i7-14700KF 3.40 GHz CPU and

Table 2
Model introduction.

Model	Layer	Neuron	Parameter
ResNet34	34	305,162	21,282,122
ResNet50	50	907,274	23,520,842
VGG13_bn	13	258,058	28,334,410
VGG16_bn	16	284,682	33,646,666
DenseNet121	121	563,210	6,956,426
DenseNet161	161	903,178	26,482,570
Inception_v3	159	4,032,362	21,638,954
MobileNet_v2	105	1,639,946	2,236,682
LeNet-1	3	4106	3246
LeNet-5	7	8094	60,850

Table 3

FDR across different manifold regions (R1 to R4) for various models on four datasets. The results demonstrate that as the manifold regions approach the decision boundary (i.e., R4), the ability of testing tool to detect model faults increases, with *CtrlFuzz* achieving the highest FDR in most cases, showing its effectiveness in revealing model vulnerabilities.

Dataset	Model	R1	R2	R3	R4
ImageNet(64 × 64)	VGG13_bn	6.5	13.0	53.0	85.0
	ResNet34	4.0	11.5	66.5	86.5
	MobileNet_v2	7.5	9.0	56.0	87.0
	VGG13_bn	8.0	12.5	54.0	88.0
TinyImageNet	ResNet34	6.0	8.5	67.5	87.5
	MobileNet_v2	5.5	11.5	57.0	86.0
	ResNet34	7.0	14.0	63.0	90.0
CIFAR-10	VGG16_bn	6.5	9.5	57.0	89.5
	Inception_v3	5.5	9.0	59.5	90.5
MNIST	LeNet-5	6.0	7.0	48.0	87.0

a NVIDIA GeForce RTX 4070 TI SUPER GPU with 16 GB of VRAM. The training setup uses the following parameters: learning rate (lr) is 3×10^{-4} , cosine noise scheduler for adding noise to the samples, batch size is 32, and unconditional probability (p_{uncond}) is 0.2.

4.2. Research questions

We empirically evaluate the performance of *CtrlFuzz* by systematically addressing three key research questions regarding its controllability, naturalness, and effectiveness.

RQ1: Does manifold-guided generation improve the controllability of DNN fuzzing tools? To clarify this, we investigated whether manifold-guided generation enhances the control of test case synthesis in Section 4.2.1.

RQ2: Does manifold-guided generation compromise the performance of generation-based fuzzing tools? To articulate this, we analyzed whether *CtrlFuzz* compromises the semantic plausibility and distributional alignment of generated tests in Section 4.2.2.

RQ3: Can *CtrlFuzz*-generated test inputs effectively expose and support the mitigation of DNN defects? To answer this, we determined whether the test cases generated by *CtrlFuzz* meaningfully improve the robustness of the target deep neural networks in Section 4.2.3.

4.2.1. Controllability

Does manifold-guided generation improve the controllability of DNN fuzzing tools?

Generation-based fuzzing tools have shown considerable success in producing test inputs that resemble real-world data. However, their inherent stochasticity limits controllability over the generated inputs. *CtrlFuzz* addresses this limitation by introducing manifold-aware guidance, allowing test cases to be synthesized along semantically meaningful directions within the learned manifold space. In particular, the

Table 4

Relative improvement in PRDC and SSIM metrics. The values represent the improvements of the *CtrlFuzz* over the traditional diffusion models (DDPM, DDIM), which reflects the performance enhancement in generating high-quality images.

Dataset	Metric	DDPM	DDIM	CtrlFuzz
ImageNet(64 × 64)	Precision	0.062	0.22	0.241
	Recall	0.041	0.161	0.192
	Density	0.022	0.356	0.279
	Coverage	0.015	0.184	0.331
	SSIM	0.0171	0.0134	0.0162
TinyImageNet	Precision	0.074	0.24	0.22
	Recall	0.038	0.13	0.27
	Density	0.031	0.265	0.332
	Coverage	0.026	0.22	0.21
	SSIM	0.0328	0.0269	0.0271
CIFAR-10	Precision	0.033	0.102	0.268
	Recall	0.091	0.127	0.155
	Density	0.067	0.085	0.088
	Coverage	0.041	0.077	0.086
	SSIM	0.0405	0.0592	0.0601

low-dimensional projection of the manifold is partitioned into multiple regions, and the generation process is evaluated for its ability to produce distinct test samples across these regions.

To assess this controllability, we define vulnerable regions as directions from the seed sample toward areas closer to the model's decision boundary. *CtrlFuzz* is designed to prioritize these regions, guiding test input generation toward boundary-adjacent zones more likely to reveal model vulnerabilities.

We evaluate the controllability by measuring the FDR across four manifold regions (R1–R4) surrounding each seed input, where R1 is closest to the seed and R4 is closest to the decision boundary. As shown in Table 3, experiments conducted on four benchmark datasets (ImageNet, TinyImageNet, CIFAR-10, and MNIST) and various DNN architectures demonstrated that *CtrlFuzz* consistently achieves significantly higher FDRs in boundary-proximal regions. For example, on ImageNet with ResNet34, the FDR increases from 3.0% in R1 to 86.5% in R4; on TinyImageNet with VGG13_bn, R4 reaches 88.0%; and on CIFAR-10 with ResNet34, R4 reaches 90.0%. Similar patterns are observed across other datasets and models. A reasonable explanation is that the test cases located near the decision boundary are in transitional zones between classes, which are the most difficult to classify. For these samples, the models tend to exhibit lower prediction confidence and higher misclassification risks, thereby making them more likely to expose vulnerabilities. This phenomenon is directly reflected in the elevated FDR values (see Fig. 4).

These results demonstrate a strong correlation between manifold-guided directional control and fault-triggering capability. By incorporating controllable generation along the manifold, *CtrlFuzz* substantially enhances the controllability of generation-based fuzzing, enabling the creation of realistic yet vulnerability-targeted test inputs for deep neural networks.

4.2.2. Naturalness

Does manifold-guided generation compromise the performance of generation-based fuzzing tools?

CtrlFuzz is designed to enhance the controllability without compromising the strength of generative models, thereby refining the efficiency and effectiveness of test input synthesis. Compared to mutation-based approaches, generation-based fuzzing is inherently more adept at capturing the underlying data distribution. An important criterion for evaluating the quality of generative models is the diversity and realism of synthesized samples.

Table 5

Comparison of FDR across several benchmark methods on four datasets and various models. The results show that *CtrlFuzz* persistently achieves higher FDR compared to baseline coverage-guided and generation-based fuzzing methods.

Models	NC			NBC	SNAC	TKNC			KMNC		DLRegion		GD	IDC	TIGs	CtrlFuzz
	0.4	0.5	0.6			1	3	5	100	1000	NC	TKNC				
ImageNet(64*64)	Inception_v3	78.5	79.0	78.0	75.0	68.0	76.5	77.5	78.0	73.5	75.0	76.0	75.0	78.0	82.5	86.0
	VGG13_bn	67.0	67.5	68.5	75.5	76.5	77.0	76.5	77.5	77.0	79.50	74.0	77.5	81.5	83.0	85.0
	ResNet34	76.5	75.5	75.5	75.0	76.0	76.0	77.5	78.5	74.5	76.0	78.0	76.5	79.0	84.0	86.5
	MobileNet_v2	78.0	76.5	77.5	56.0	77.0	76.0	77.0	77.5	77.0	80.5	76.5	75.0	83.5	85.5	87.0
	ResNet50	77.0	78.0	77.5	74.0	67.5	75.5	77.0	77.0	75.5	82.0	77.0	78.0	73.5	78.5	81.0
	DenseNet121	77.5	78.0	77.0	55.0	77.0	76.5	77.5	78.5	72.0	77.5	73.0	72.0	77.0	75.0	80.5
TinyImageNet	Inception_v3	77.5	76.5	78.0	74.0	67.5	76.0	78.0	78.5	77.0	79.0	77.0	75.0	82.0	79.5	86.5
	VGG13_bn	66.0	67.0	68.0	76.0	78.5	77.0	77.5	79.0	75.5	78.0	77.5	76.0	85.5	82.0	88.0
	ResNet34	78.5	77.0	76.5	74.5	77.5	76.0	76.5	77.5	76.0	80.5	75.0	76.5	81.0	80.5	87.5
	MobileNet_v2	76.0	78.0	76.5	55.0	78.0	75.0	76.0	78.0	73.5	77.0	74.0	75.0	78.5	77.5	86.0
	ResNet50	78.0	76.5	77.5	74.5	65.0	75.5	75.5	76.5	78.0	78.5	76.0	77.0	76.0	75.5	85.5
	DenseNet121	75.5	78.0	76.0	56.0	76.5	76.0	77.5	77.5	74.0	80.0	73.0	78.0	82.0	79.0	86.5
CIFAR-10	ResNet34	58.5	60.0	59.5	78.0	62.5	57.5	59.0	59.5	77.5	79.0	57.0	56.0	87.5	65.5	90.0
	ResNet50	56.0	57.5	57.0	77.5	64.0	56.0	57.5	57.0	73.0	76.5	55.0	62.0	88.0	68.0	89.0
	VGG13_bn	59.0	58.5	61.0	79.5	61.0	58.0	57.5	58.0	77.0	81.0	54.0	55.0	86.5	63.5	88.5
	VGG16_bn	57.5	58.0	59.5	80.0	61.5	57.0	58.5	56.0	79.0	81.5	53.0	52.0	88.0	70.0	89.5
	DenseNet121	60.0	59.5	61.5	78.5	63.0	59.5	59.0	60.0	76.5	78.0	63.0	61.0	88.5	67.5	90.5
	DenseNet161	58.0	59.5	60.5	80.5	64.0	58.5	60.0	57.5	74.0	77.5	62.0	64.0	87.5	66.0	89.0
	Inception_v3	55.5	57.0	59.0	77.5	62.5	55.0	57.0	58.5	76.0	80.0	65.0	67.0	88.0	69.5	90.5
	MobileNet_v2	57.0	57.5	58.5	80.0	60.0	57.5	59.0	59.0	75.0	82.0	57.0	59.0	89.0	73.0	91.5
MNIST	LeNet-1	45.5	48.0	49.5	44.5	33.5	45.5	46.5	48.0	61.0	66.5	35.0	80.0	86.0	69.0	88.5
	LeNet-5	48.5	50.0	50.5	42.5	49.5	41.5	43.0	44.5	65.0	68.0	64.0	64.0	83.5	70.5	87.0

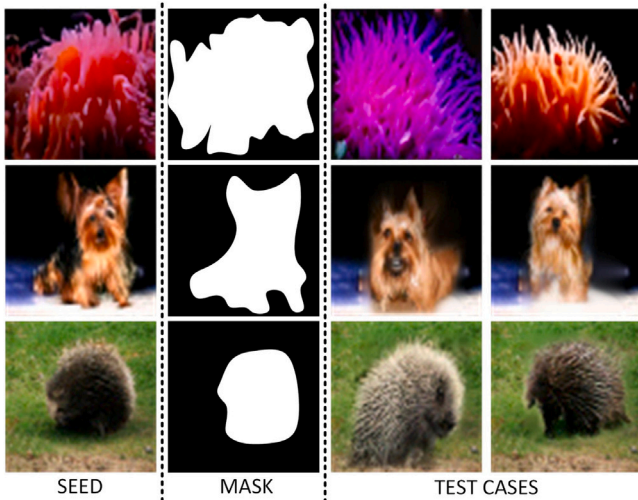


Fig. 4. The figure illustrates the generation results of *CtrlFuzz*, where the mask effectively guides the diffusion model to perform object-aware generation within specified regions.

In order to assess if *CtrlFuzz* preserves the advantages of generation-based fuzzing under manifold guidance, we compare it with baseline diffusion models (i.e., DDPM and DDIM) via PRDC and SSIM. As shown in Table 4, *CtrlFuzz* outperforms both DDPM and DDIM on almost overall indicators. For Precision, *CtrlFuzz* achieves improvements of 0.241 on ImageNet, 0.22 on TinyImageNet, and 0.268 on CIFAR-10. In terms of Recall, *CtrlFuzz* attains a 0.27 increase on TinyImageNet and exhibits consistent gains on ImageNet and CIFAR-10, which shows a wider coverage of the data distribution. With respect to Density, *CtrlFuzz* shows improvements of 0.279 on ImageNet and 0.332 on TinyImageNet, implying higher concentration within the learned feature space. For Coverage, *CtrlFuzz* delivers solid enhancements, including a 0.331 increase on ImageNet, significantly outperforming DDPM (0.015) and DDIM (0.184), and demonstrating the ability to generate representative samples.

Although improvements in SSIM are generally modest, *CtrlFuzz* still achieves superior structural similarity on CIFAR-10 and TinyImageNet, demonstrating stronger alignment with real image structures.

Overall, these results confirm that manifold guidance does not weaken the generative performance of diffusion-based fuzzing tools. Instead, *CtrlFuzz* maintains high quality of generating test cases in terms of semantic consistency, diversity, coverage, and structural similarity, which highlights the manifold-aware guidance can further strengthen the generative process rather than limiting it.

4.2.3. Effectiveness

Can *CtrlFuzz*-generated test inputs effectively expose and support the mitigation of DNN defects?

To evaluate the effectiveness of *CtrlFuzz*, we address the research question from two following key aspects: (1) Does *CtrlFuzz* generate test cases that more efficient in triggering failures of Mut? (2) Can the synthesized inputs enhance the robustness via model retraining? For the first sub-question, we measured the FDR, a widely adopted metric that reflects the proportion of test inputs capable of inducing incorrect model behaviors. Under constrained computational budgets and fixed size of test suite, a higher FDR indicates stronger capability in exposing model defects.

Table 5 presents FDR results for multiple mainstream DNN architectures (e.g., ResNet, VGG, MobileNet) across four benchmark datasets (ImageNet, TinyImageNet, CIFAR-10, MNIST) under various testing methods. A higher FDR signifies a test suite's stronger fault exposure capacity. We compared *CtrlFuzz* against representative neuron coverage-guided fuzzing tools (NC, NBC, SNAC, TKNC), their recent enhancement (DLRegion), and distance-guided methods (GD).

As shown in Fig. 5, *CtrlFuzz* consistently outperforms all baselines across most model-dataset pairs, particularly as the test suite size increases. On CIFAR-10, *CtrlFuzz* achieves an FDR of 91.5% on MobileNet_v2, markedly higher than NC (58.5%) and TKNC (60.0%). On ImageNet, the FDR of ResNet50 increases from 75%–78% (baseline) to 87.0% with *CtrlFuzz*. Similar advantages are observed on TinyImageNet (88.0% FDR on VGG13_bn) and MNIST (88.5% on LeNet-5). We have

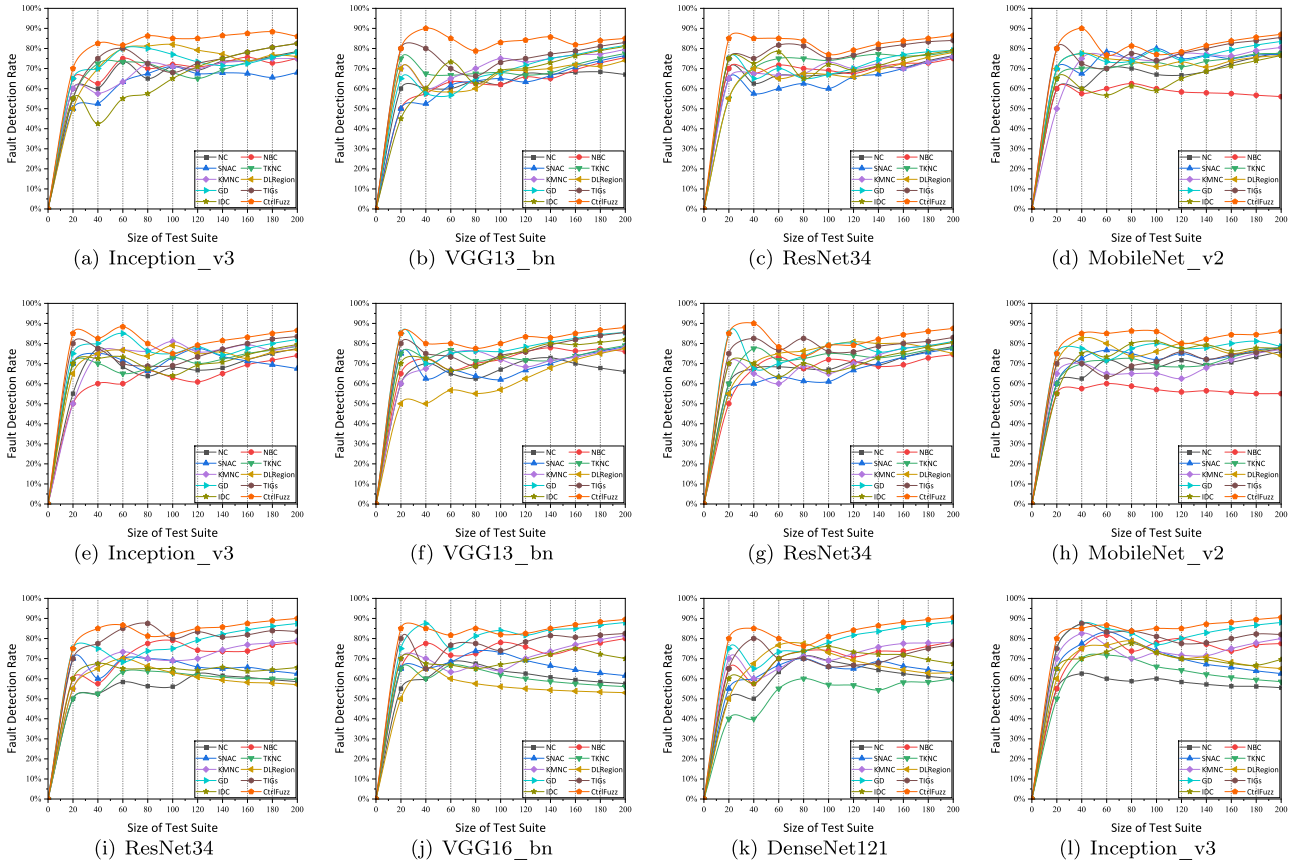


Fig. 5. We evaluate the effectiveness of different CGF methods in identifying erroneous behaviors on four MuTs. The first row represents ImageNet (64×64) dataset, the second row represents TinyImageNet dataset, and the third row represents the CIFAR-10 dataset. Our method displays a substantially superior ability to detect erroneous behaviors in comparison to baselines.

Table 6

The retraining experiments on each MuT.

MuTs	ResNet18	ResNet34	VGG13_bn	VGG16_bn	MobileNet_V2	DenseNet121	DeneNet161	GoogLeNet	LeNet5
Before	93.57%	92.86%	94.16%	94.11%	93.56%	93.79%	93.68%	92.56%	98.79%
After	94.03%	93.23%	94.62%	94.58%	93.76%	94.02%	94.06%	92.76%	98.98%
Improved	0.46%	0.37%	0.46%	0.47%	0.20%	0.23%	0.38%	0.20%	0.19%

observed that *CtrlFuzz* demonstrates a advantage over TIGs on models with lower complexity, with a 40% improvement in FDR on LeNet-1. In contrast, IDC exhibits relatively consistent performance across models and datasets of different scales.

These findings confirm that the manifold-guided diffusion strategy of *CtrlFuzz* empowers systematic generation of test inputs that are more effective in revealing model defects. We attribute this advantage to *CtrlFuzz*'s hybrid design, which combines the strengths of mutation- and generation-based fuzzing, yielding semantically diverse, distributional alignment, and controllable test cases. Furthermore, different from the traditional methods, *CtrlFuzz* is agnostic to internal model structure or information. Though TMRC module in *CtrlFuzz* can be integrated into any mutation-based CGF, its effectiveness will still be constrained by the ability of mutators to achieve sufficient coverage.

4.3. Model retraining

The retraining of MuT promotes the model robustness by augmenting the training set with error-inducing inputs, which allows the MuT to learn from previously erroneous behaviors. It is widely recognized as an effective strategy for refining the reliability of DNNs, and adopted in the evaluation of testing tools to assess the practical utility of the generated test cases [59].

In the adversarial training, we integrate the failure-triggering test inputs generated by *CtrlFuzz* into the original training set and retrain each MuT for 10 epochs to evaluate whether these samples contribute to performance improvement. As shown in Table 6, the retraining results demonstrate that *CtrlFuzz* has a consistently positive effect on robustness of MuT. On average, the test accuracy of all MuTs increased by approximately 0.39%, showing enhanced generalization and resistance to adversarial behaviors (see Fig. 6).

Explicitly, high-capacity models such as ResNet18, VGG13.bn, and VGG16.bn experienced accuracy gains of 0.46–0.47%, despite already achieving strong baseline performance ($\geq 94\%$). Given the high performance and dataset size, the improvements are considerable. Alternatively, smaller gains were observed for more concise models such as MobileNet_v2 and GoogLeNet (around 0.20%), potentially due to differences in parameter redundancy and model capacity to internalize correction.

For lightweight architectures like LeNet-5, where baseline performance was already high, the improvement from *CtrlFuzz* was more modest. Nonetheless, the consistent upward trend across distinct models reinforces the value of integrating *CtrlFuzz*-generated samples into the retraining workflow.

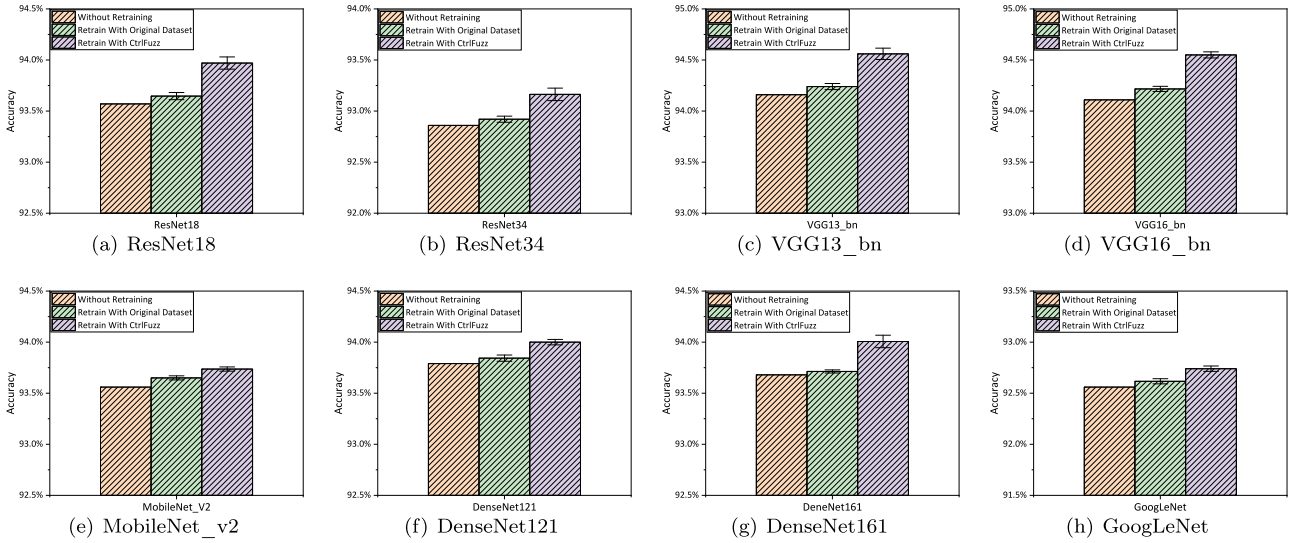


Fig. 6. We conduct a retraining process over 10 epochs using the MuTs. The experimental findings indicate that *CtrlFuzz* significantly improved the model's performance, with an average accuracy increase of 0.39%.

Table 7

Ablation study results on CIFAR-10 dataset. The table compares the FDR for the models of full *CtrlFuzz*, random mutation, and whether the manifold guidance, diffusion model, and semantic decomposition modules were removed.

Model	Without TMRC	Without diffusion	Without mask	Random mutation	CtrlFuzz
ResNet34	8.50%	43.50%	78.50%	22.50%	90.00%
VGG13_bn	7.50%	36.00%	83.00%	34.00%	88.50%
DenseNet121	7.00%	51.50%	84.00%	29.50%	90.50%
MobileNet_v2	11.50%	39.50%	82.50%	31.50%	91.50%

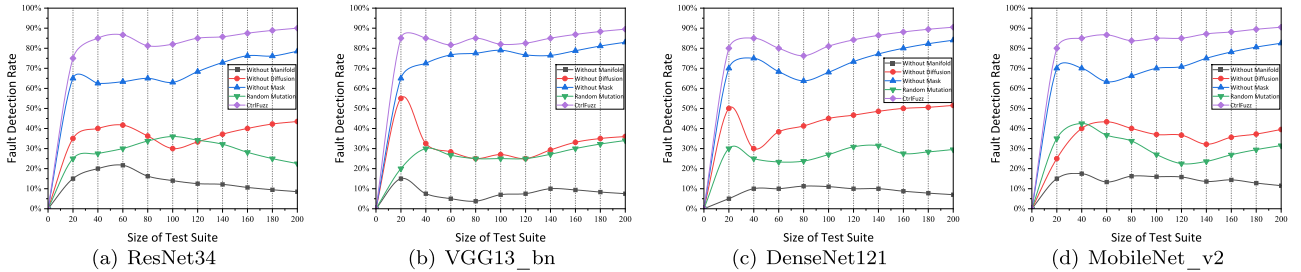


Fig. 7. Ablation study results for different models on the CIFAR-10 dataset. The Fig. shows the FDR results of ResNet34, VGG13_bn, DenseNet121, and MobileNet_v2 including the full *CtrlFuzz* and ablated versions.

5. Ablation study

We performed ablation studies to demonstrate contribution of each module in *CtrlFuzz*. By removing the two core components one by one (i.e., the manifold guidance module and the diffusion-based generation module), we evaluated four representative MuTs (ResNet34, VGG13_bn, DenseNet121, and MobileNet_v2) on the CIFAR-10 dataset with FDR. The ablation experiments are configured with four settings: the full *CtrlFuzz* implementation, a variant excluding the manifold guidance module, a variant removing the diffusion generation module, a variant removing mask, and a variant applying random mutators.

As illustrated in Fig. 7, the system maintains reasonable performance when eliminating the manifold guidance module (e.g., 43.5% FDR for ResNet34 and 36.0% for VGG13_bn), indicating the diffusion model's inherent capability in test case generation. Nevertheless, the absence of manifold guidance prevents generated inuts from effectively approaching the vulnerable regions on manifold, resulting in lower FDR compared to the full *CtrlFuzz*. On the other hand, removing the diffusion module while remaining manifold guidance with random mutation leads to more significant performance degradation. For instance, ResNet34 and VGG13_bn show FDR decline to 8.5%

and 7.5%, respectively. This phenomenon demonstrates that random mutation struggle to synthesize realistic and challenging test inputs within limited computational resources, even when guided in manifold space.

The full *CtrlFuzz* implementation achieves optimal performance on all models. As shown in Table 7, it attains 91.5% FDR on MobileNet_v2 and surpasses 90% on both ResNet34 and DenseNet121. Compared to the two ablated versions ("Without Manifold" and "Without Diffusion"), the FDR improvement exceeds 80% on multiple models, strongly validating the synergistic effect between manifold guidance and diffusion process. To be specific, the diffusion model guarantees naturalness and diversity in test case generation, while the manifold guidance precisely locates the vulnerable regions on manifold. The ablated modules of *CtrlFuzz* incorporate the two aforementioned functionalities respectively.

Our ablation studies conclusively demonstrate that the integration of manifold guidance and diffusion generation constitutes the foundation of *CtrlFuzz*'s superior effectiveness. This combination enables *CtrlFuzz* to generate controllable, semantically rich test cases with high defect-revealing capability, demonstrating significant advancement in

Table 8

The time consumption of generating single test case for testing a ResNet18 model with CIFAR-10.

Time cost per case (s)				
NC	NBC	SNAC	TKNC	TKNP
2.46	2.63	2.41	4.33	56.33
KMNC	DLRegion	GD	STD	CtrlFuzz
10.09	3.77	3.28	3.05	7.73

deep neural network testing methodologies. Conversely, black-box testing methods demonstrate lower time consumption but exhibit limited efficacy in improving structural coverage.

5.1. Time consumption

In CtrlFuzz, the linear interpolation step (Algorithm 1) operates over T steps. Each step involves a generation step and semantic distance estimation in a low-dimensional space. If the input is an N -dimensional vector, the overall time complexity is $O(T \times N)$, which ensures the algorithm itself remains computationally efficient. The dominant cost arises from the computation time of the generative model (e.g., a diffusion model), which depends on model size and image resolution.

To assess the efficiency of CtrlFuzz, we performed a comparative analysis of time consumption involved different testing methods. As demonstrated in Table 7, the time overhead of CtrlFuzz is lower than KMNC and TKNP-guided testing tools but slightly higher than other methods. The possible increased time cost associated with KMNC and TKNP is due to the dependence on computationally demanding analyses of neuron influence and functional intervals. Conversely, black-box testing methods demonstrate lower time consumption but exhibit limited efficacy in improving structural coverage due to the absence of internal information of MuT. CtrlFuzz effectively balances the trade-off, enhancing both the error-detection capabilities and the extent of test coverage within a reasonable time consumption (see Table 8).

6. Conclusion

In this work, we present the CtrlFuzz, a controllable, manifold-aware diffusion model-based fuzz testing tool for detecting defects of DNNs. CtrlFuzz accomplishes two significant advances: (1) it enhances the controllability of generation-based CGF via directing test inputs along with targeted manifold; (2) it integrates the strengths of mutation-based and generation-based CGF, facilitating semantically meaningful and distribution-aligned test case synthesis. Empirical findings from 4 datasets and 10 comprehensive models indicate that CtrlFuzz not only improves fault detection rates but also enhances model robustness through retraining. CtrlFuzz provides a precise and effective solution for systematic, controllable fuzzing process. In our future work, we will further refine and integrate manifold coverage with generative models to augment the interpretability of DNN fuzz testing.

CRedit authorship contribution statement

Aoshuang Ye: Writing – review & editing, Writing – original draft, Project administration, Methodology, Investigation. **Shilin Zhang:** Writing – original draft, Software, Methodology. **Runze Yan:** Software, Methodology. **Jianpeng Ke:** Validation, Investigation, Data curation. **Fei Zhu:** Supervision, Funding acquisition. **Benxiao Tang:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Hubei Provincial Natural Science Foundation of China (Grant No. Q20231714 and No. 2025AFB323) and the Open Fund of Advanced Cryptography and System Security Key Laboratory of Sichuan Province, China (Grant No. SKLACSS-202313).

Data availability

No data was used for the research described in the article.

References

- [1] Y. Zhao, Y. Zhu, L. Zhao, J. Huang, Q. Zhi, Behavioral decision-making and safety verification approaches for autonomous driving system in extreme scenarios, *J. Syst. Softw.* 226 (2025) 112385.
- [2] P. Yin, J. Song, Y. Bouteraa, L. Qian, D. Martín, M. Khishe, A cognitive few-shot learning for medical diagnosis: A case study on cleft lip and palate and Parkinson's disease, *Expert Syst. Appl.* 262 (2025) 125713.
- [3] X. Liu, W. Jiang, Z. Li, X. Jin, Z. Ma, Q. Li, FuzzAGG: A fuzzing-driven attack graph generation framework for industrial robot systems, *Comput. Secur.* 150 (2025) 104223.
- [4] Z. Liu, Y. Feng, Y. Yin, Z. Chen, DeepState: Selecting test suites to enhance the robustness of recurrent neural networks, in: 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25–27, 2022, ACM, 2022, pp. 598–609.
- [5] H. Zhou, W. Li, Z. Kong, J. Guo, Y. Zhang, B. Yu, L. Zhang, C. Liu, DeepBillboard: systematic physical-world testing of autonomous driving systems, in: ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June – 19 July, 2020, ACM, 2020, pp. 347–358.
- [6] Z. Wang, J. Guo, Y. Chen, F. She, The effect of combinatorial coverage for neurons on fault detection in deep neural networks, in: 2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C), IEEE, 2021, pp. 77–82.
- [7] A. Odena, C. Olsson, D.G. Andersen, L.J. Goodfellow, TensorFuzz: Debugging neural networks with coverage-guided fuzzing, in: K. Chaudhuri, R. Salakhutdinov (Eds.), Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9–15 June 2019, Long Beach, California, USA, in: Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 4901–4911.
- [8] J. Guo, Y. Jiang, Y. Zhao, Q. Chen, J. Sun, Dfuzz: differential fuzzing testing of deep learning systems, in: G.T. Leavens, A. Garcia, C.S. Pasareanu (Eds.), Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04–09, 2018, ACM, 2018, pp. 739–743.
- [9] Y. Yuan, Q. Pang, S. Wang, Revisiting neuron coverage for DNN testing: A layer-wise and distribution-aware criterion, in: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023, IEEE, 2023, pp. 1200–1212.
- [10] X. Xie, L. Ma, F. Juefei-Xu, M. Xue, H. Chen, Y. Liu, J. Zhao, B. Li, J. Yin, S. See, DeepHunter: a coverage-guided fuzz testing framework for deep neural networks, in: D. Zhang, A. Möller (Eds.), Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15–19, 2019, ACM, 2019, pp. 146–157.
- [11] L. Huang, W. Sun, M. Yan, Z. Liu, Y. Lei, D. Lo, Neuron semantic-guided test generation for deep neural networks fuzzing, *ACM Trans. Softw. Eng. Methodol.* 34 (1) (2025) 15:1–15:38.
- [12] C. Tao, Y. Tao, H. Guo, Z. Huang, X. Sun, DLRegion: Coverage-guided fuzz testing of deep neural networks with region-based neuron selection strategies, *Inf. Softw. Technol.* 162 (2023) 107266.
- [13] S. Bond-Taylor, A. Leach, Y. Long, C.G. Willcocks, Deep generative modelling: A comparative review of VAEs, GANs, normalizing flows, energy-based and autoregressive models, *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (11) (2022) 7327–7347.
- [14] M. Zhang, Y. Zhang, L. Zhang, C. Liu, S. Khurshid, DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems, in: M. Huchard, C. Kästner, G. Fraser (Eds.), Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3–7, 2018, ACM, 2018, pp. 132–142.
- [15] S. Missaoui, S. Gerasimou, N. Matragkas, Semantic data augmentation for deep learning testing using generative AI, in: 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11–15, 2023, IEEE, 2023, pp. 1694–1698.
- [16] Z. Li, X. Wu, D. Zhu, M. Cheng, S. Chen, F. Zhang, X. Xie, L. Ma, J. Zhao, Generative model-based testing on decision-making policies, in: 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11–15, 2023, IEEE, 2023, pp. 243–254.

- [17] J. Kim, R. Feldt, S. Yoo, Guiding deep learning system testing using surprise adequacy, in: J.M. Atlee, T. Bultan, J. Whittle (Eds.), Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019, IEEE / ACM, 2019, pp. 1039–1049.
- [18] Z. Li, Z. Xu, R. Ji, M. Pan, T. Zhang, L. Wang, X. Li, Distance-aware test input selection for deep neural networks, in: M. Christakis, M. Pradel (Eds.), Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024, ACM, 2024, pp. 248–260.
- [19] Z. Aghababaeian, M. Abdellatif, M. Dadkhah, L.C. Briand, DeepGD: A multi-objective black-box test selection approach for deep neural networks, ACM Trans. Softw. Eng. Methodol. 33 (6) (2024) 158.
- [20] B.T. Kiani, J. Wang, M. Weber, Hardness of learning neural networks under the manifold hypothesis, in: A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J.M. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024, 2024.
- [21] S. Demir, H.F. Eniser, A. Sen, DeepSmartFuzzer: Reward guided test generation for deep learning, in: H. Espinoza, J.A. McDermid, X. Huang, M. Castillo-Effen, X.C. Chen, J. Hernández-Orallo, S.O. Hfegartaigh, R. Mallah (Eds.), Proceedings of the Workshop on Artificial Intelligence Safety 2020 Co-located with the 29th International Joint Conference on Artificial Intelligence and the 17th Pacific Rim International Conference on Artificial Intelligence (IJCAI-PRICAI 2020), Yokohama, Japan, January, 2021, in: CEUR Workshop Proceedings, vol. 2640, CEUR-WS.org, 2020.
- [22] Y. Tian, K. Pei, S. Jana, B. Ray, DeepTest: automated testing of deep-neural-network-driven autonomous cars, in: M. Chaudron, I. Crnkovic, M. Chechik, M. Harman (Eds.), Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018, ACM, 2018, pp. 303–314.
- [23] Y. Yuan, Q. Pang, S. Wang, Unveiling hidden DNN defects with decision-based metamorphic testing, in: 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022, ACM, 2022, pp. 113:1–113:13.
- [24] M. Usman, Y. Sun, D. Gopinath, R. Dange, L. Manolache, C.S. Pasareanu, An overview of structural coverage metrics for testing neural networks, Int. J. Softw. Tools Technol. Transf. 25 (3) (2023) 393–405.
- [25] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, Adv. Neural Inf. Process. Syst. 27 (2014).
- [26] D.P. Kingma, Auto-encoding variational bayes, 2013, arXiv preprint arXiv:1312.6114.
- [27] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, S. Ganguli, Deep unsupervised learning using nonequilibrium thermodynamics, in: International Conference on Machine Learning, PMLR, 2015, pp. 2256–2265.
- [28] W. Huang, X. Zhao, A. Banks, V. Cox, X. Huang, Hierarchical distribution-aware testing of deep learning, ACM Trans. Softw. Eng. Methodol. 33 (2) (2024) 42:1–42:35.
- [29] K. Pei, Y. Cao, J. Yang, S. Jana, DeepXplore: automated whitebox testing of deep learning systems, Commun. ACM 62 (11) (2019) 137–145.
- [30] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, Y. Wang, DeepGauge: multi-granularity testing criteria for deep learning systems, in: M. Huchard, C. Kästner, G. Fraser (Eds.), Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018, ACM, 2018, pp. 120–131.
- [31] Z. Ji, P. Ma, Y. Yuan, S. Wang, CC: causality-aware coverage criterion for deep neural networks, in: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023, IEEE, 2023, pp. 1788–1800.
- [32] Y. Shi, B. Yin, J. Shi, Markov model based coverage testing of deep learning software systems, Inf. Softw. Technol. 179 (2025) 107628.
- [33] F. Harel-Canada, L. Wang, M.A. Gulzar, Q. Gu, M. Kim, Is neuron coverage a meaningful measure for testing deep neural networks? in: P. Devanbu, M.B. Cohen, T. Zimmermann (Eds.), ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020, ACM, 2020, pp. 851–862.
- [34] L. Wang, X. Xie, X. Du, M. Tian, Q. Guo, Z. Yang, C. Shen, DistXplore: Distribution-guided testing for evaluating and enhancing deep learning systems, in: S. Chandra, K. Blincoe, P. Tonella (Eds.), Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023, ACM, 2023, pp. 68–80.
- [35] J. Kim, R. Feldt, S. Yoo, Evaluating surprise adequacy for deep learning system testing, ACM Trans. Softw. Eng. Methodol. 32 (2) (2023) 42:1–42:29.
- [36] S. Dola, R. McDaniel, M.B. Dwyer, M.L. Soffa, CIT4DNN: generating diverse and rare inputs for neural networks using latent space combinatorial testing, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024, ACM, 2024, pp. 118:1–118:13.
- [37] Q. Zhu, S. Wu, S. Fang, Q. Wu, S. Xie, S.S. Agaian, Fast tensor robust principal component analysis with estimated multi-rank and Riemannian optimization, Appl. Intell. 55 (1) (2025) 52.
- [38] M. Allaoui, S.B. Belhaouiari, R. Hedjam, K. Bouanane, M.L. Kherfi, T-SNE-PSO: Optimizing t-SNE using particle swarm optimization, Expert Syst. Appl. 269 (2025) 126398.
- [39] H. Li, M. Trocan, Sparse reconstruction of ISOMAP representations, J. Intell. Fuzzy Syst. 37 (6) (2019) 7519–7536.
- [40] Y. Tao, Y. Li, F. Kong, Y. Shi, M. Yang, J. Yu, H. Zhang, Privacy-preserving outsourcing scheme of face recognition based on locally linear embedding, Comput. Secur. 144 (2024) 103931.
- [41] J. Liu, M. Feng, X. Xiu, W. Liu, Towards robust and sparse linear discriminant analysis for image classification, Pattern Recognit. 153 (2024) 110512.
- [42] J. Sohl-Dickstein, E.A. Weiss, N. Maheswaranathan, S. Ganguli, Deep unsupervised learning using nonequilibrium thermodynamics, in: F.R. Bach, D.M. Blei (Eds.), Proceedings of the 32nd International Conference on Machine Learning, ICLR 2015, Lille, France, 6-11 July 2015, in: JMLR Workshop and Conference Proceedings, vol. 37, JMLR.org, 2015, pp. 2256–2265.
- [43] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, Adv. Neural Inf. Process. Syst. 33 (2020) 6840–6851.
- [44] J. Song, C. Meng, S. Ermon, Denoising diffusion implicit models, 2020, arXiv preprint arXiv:2010.02502.
- [45] G. Khromov, S.P. Singh, Some fundamental aspects about Lipschitz continuity of neural networks, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net, 2024.
- [46] K. He, G. Gkioxari, P. Dollár, R.B. Girshick, Mask R-CNN, in: IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017, IEEE Computer Society, 2017, pp. 2980–2988.
- [47] T. Byun, S. Rayadurgam, Manifold for machine learning assurance, in: G. Rothmel, D. Bae (Eds.), ICSE-NIER 2020: 42nd International Conference on Software Engineering, New Ideas and Emerging Results, Seoul, South Korea, 27 June - 19 July, 2020, ACM, 2020, pp. 97–100.
- [48] S. Dola, M.B. Dwyer, M.L. Soffa, Input distribution coverage: Measuring feature interaction adequacy in neural network testing, ACM Trans. Softw. Eng. Methodol. 32 (3) (2023) 81:1–81:48.
- [49] A. van den Oord, N. Kalchbrenner, K. Kavukcuoglu, Downsampling ImageNet 64x64, URL: <http://image-net.org/small/download.php>.
- [50] Y. Le, X. Yang, Tiny imagenet visual recognition challenge, CS 231N 7 (7) (2015) 3.
- [51] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016, IEEE Computer Society, 2016, pp. 770–778.
- [52] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, 2015.
- [53] G. Huang, Z. Liu, L. van der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017, IEEE Computer Society, 2017, pp. 2261–2269.
- [54] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 2818–2826.
- [55] M. Sandler, A.G. Howard, M. Zhu, A. Zhmoginov, L. Chen, MobileNetV2: Inverted residuals and linear bottlenecks, in: 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018, Computer Vision Foundation / IEEE Computer Society, 2018, pp. 4510–4520.
- [56] J. Xu, H. Le, Generating representative samples for few-shot classification, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 9003–9013.
- [57] D. Brunet, E.R. Vrsay, Z. Wang, On the mathematical properties of the structural similarity index, IEEE Trans. Image Process. 21 (4) (2011) 1488–1499.
- [58] Maryam, M. Biagiola, A. Stocco, V. Riccio, Benchmarking generative AI models for deep learning test input generation, in: IEEE Conference on Software Testing, Verification and Validation, ICST 2025, Napoli, Italy, March 31 - April 4, 2025, IEEE, 2025, pp. 174–185.
- [59] S. Yan, G. Tao, X. Liu, J. Zhai, S. Ma, L. Xu, X. Zhang, Correlations between deep neural network model coverage criteria and model quality, in: P. Devanbu, M.B. Cohen, T. Zimmermann (Eds.), ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020, ACM, 2020, pp. 775–787.